

Audion Disk Tools - пользовательское руководство

Актуально для текущего состояния проекта на 2026-07-06.

Audion Disk Tools - portable-first набор Windows-инструментов для файловых маршрутов: аудит, compare, one-way copy, BACKUP-MIRROR, two-way sync, архивация, передача по сети, SMB-share и native RClone-операции. GUI работает как Workbench над тем же CLI: пользователь выбирает **Источник**, **Назначение**, режим и параметры, а команда выполняется через portable runtime с живым терминалом справа.

Главная идея проекта: **Источник** и **Назначение** больше не обязаны быть проектными **input** и **output**. Это могут быть внешние диски, большие каталоги, UNC-пути, SMB-шары, папки облачных клиентов или обычные локальные директории.

Документация

- [README_RU.md](#) - краткая карта возможностей.
- [USER_GUIDE_EN.md](#) - английская версия руководства.
- [AUDION_DISK_TOOLS_RU.md](#) - архитектура, папки, конфиги.
- [WORKBENCH_AND_GUI_RU.md](#) - GUI Workbench, терминал, кэши, tooltip.
- [COMMAND_REFERENCE_RU.md](#) - полный справочник команд и параметров.
- [SYNC_PROFILES_RU.md](#) - профили, пресеты, фильтры расширений.
- [ARCHIVE_OPERATIONS_RU.md](#) - архивы, SFX, шифрование.
- [TRANSFER_RU.md](#) - трансфер данных: операции, движки, маски, упаковка перед отправкой.
- [RCLONE_OPERATIONS_RU.md](#) - подключения, хранилище, диагностика, шифрование конфига.
- [RCLONE_PORTABILITY_RU.md](#) - установка RClone, portable/system config, import/export.
- [MANIFEST_REFERENCE_RU.md](#) - Manifest / Diff, списки относительных путей и служебные **__CHECKSUMS__.b3**.
- [PDF_EXPORT_RU.md](#) - конверсия документации в PDF под **docs\PDF**.

- [KNOWN_PITFALLS_RU.md](#) - типовые ошибки и безопасные привычки.
- [SMOKE_TEST_CHECKLIST_RU.md](#) - проверка перед релизом или после крупных изменений.

Запуск

Основной GUI:

```
launcher_gui.cmd
```

GUI с правым терминалом стартует только на `127.0.0.1`, `localhost` или `::1`. Non-loopback host блокируется, если явно не задан `AUDION_ALLOW_REMOTE_GUI=1`. Desktop wrapper не просит UAC сам; для осознанного admin-запуска используйте `--elevate` или `AUDION_ELEVATE=1`.

Если нужен CLI/FZF launcher:

```
launcher_project.cmd  
launcher_project_ru.cmd  
launcher_profiles.cmd  
launcher_profiles_ru.cmd
```

Если нужно запустить CLI напрямую:

```
runtime\python.exe system_core\main.py info  
runtime\python.exe system_core\main.py pairs
```

Если runtime ещё не собран:

```
builder_main.cmd
```

Рабочая модель

1. Выберите `Источник`.
2. Выберите `Назначение`.
3. Выберите ROOT-команду или сохранённый профиль.
4. При необходимости задайте include/exclude маски.
5. Для рискованных операций сначала запустите compare или dry-run.

6. Смотрите правый терминал, прогресс, отчёты в **report** и логи в **logs**.

Источник - сторона истины для one-way и mirror-сценариев. **Назначение** - принимающая или backup-сторона. Для RClone upload **Источник** копируется в remote. Для RClone download remote копируется в **Назначение**.

Основные режимы

BACKUP-MIRROR копирует и обновляет файлы из источника в цель, проверяет ранние фазы и затем удаляет target-only файлы. Это hard mirror. Используйте его только когда цель действительно должна стать зеркалом источника.

Односторонняя копирует новые и изменённые файлы из источника в цель. Лишние файлы в цели остаются жить.

Two-Way sync обменивает отсутствующие или более новые файлы в обе стороны. Автоматических удалений нет.

Маски - ручной конструктор glob-фильтра. Он может запускать **BACKUP MIRROR**, **ONE-WAY** или **TWO-WAY**. Для **BACKUP MIRROR** в этом конструкторе используется безопасный **mirror_safe**: target-only файлы переносятся в **_audion_quarantine**, а не удаляются hard-delete.

Сохранённые профили берутся из **config\sync_pairs.json**. Если файл пустой, GUI показывает bundled example-профили из **config\sync_pairs.example.json**, чтобы интерфейс оставался обучающим и пригодным к экспорту.

Архивация упаковывает выбранный источник в выбранную цель. Основная линейка форматов: ZIP, 7Z, SFX, TAR, TAR.GZ и TAR.ZSTD, если соответствующий backend доступен. TAR.LZ4 остаётся дополнительным форматом и появляется только при найденном **lz4.exe**.

ТРАНСФЕР ДАННЫХ - один раздел на папку, шару, сервер и облако: пять операций, список машин-назначений, движок под каждую пару выбирается сам — Robocopy или RClone. Рядом там же хранилище, подключения и диагностика.

Manifest / Diff работает по текущим Workbench **Источник -> Назначение**: умеет сузить расчёт по группам расширений, построить Diff и сразу применить его зеркалом или односторонним прогоном. Отдельные **__CHECKSUMS__.b3** для источника/цели остаются служебной проверкой.

Параметры безопасности

`mode=safe` - дефолт. Сравнение строится осторожнее, чем `quick`, и подходит для реальных рабочих операций.

`mode=quick` - быстрее, но больше полагается на метаданные.

`mode=strict` доступен в CLI для части команд и нужен для максимально строгих сравнений.

`operation_policy=copy_update` - копировать/обновлять, target-only оставить.

`operation_policy=mirror_safe` - target-only переносить в `_audion_quarantine`.

`operation_policy=mirror_hard` - target-only удалять после успешных ранних фаз. Для отфильтрованного hard mirror и большого delete ratio CLI требует явные guard-флаги.

`anchor=true` в профиле требует файл-якорь `.audion-anchor` в обоих корнях до сканирования. Это нужно там, где корень приходит из сети: отключившийся монтаж выглядит как существующая пустая папка, и без якоря прогон читается как "удалить в приёмнике всё". Якорь создаётся один раз отдельной командой и никогда не создаётся сам во время прогона:

```
runtime\python.exe system_core\main.py anchor N:\Projects
```

Над полями команды GUI показывает карточку **ПРОВЕРКА ПЕРЕД ЗАПУСКОМ**: якорь, свободное место против планируемой записи и планируемые удаления числом и долей от приёмника. Для зеркальных политик там же появляется токен подтверждения, привязывающий запуск к просмотренному плану. Подробно в [SYNC_PROFILES_RU.md](#).

Фильтры

Фильтр состоит из include и exclude правил:

- `include_globs` или `--mask-globs` ограничивают рабочий набор файлов.
- `exclude_globs` вычитают лишнее из выбранного набора.
- `include_presets` и `exclude_presets` ссылаются на `config\sync_presets.json`.
- `exclude_dirs` убирает каталоги вроде `.git`, `node_modules`, `cache`.
- `exclude_hidden` исключает dot-prefixed пути.

В GUI ручной выбор расширений имеет вкладки **ВКЛЮЧИТЬ** и **ИСКЛЮЧИТЬ**. Пустой выбор означает: не добавлять runtime override и оставить профиль как есть.

RClone в ежедневной работе

Для rclone используется найденный `rclone.exe`, обычно `Tools\rclone\rclone.exe`. Линейка `Installer RClone` даёт две маленькие кнопки: `System` ставит user-scope RClone в `%LOCALAPPDATA%\Programs\rclone` и добавляет его в user PATH, `Portable` ставит/обновляет `Tools\rclone`. Оба installer-а проверяют `rclone.exe version` и не перезаписывают `rclone.conf`.

Нормальный порядок первого запуска:

1. Нажмите install, если `rclone.exe` ещё не найден.
2. Запустите `Version`.
3. Для Yandex/Google Drive/OneDrive/Dropbox/Box/Mega используйте `Провайдеры OAuth`: GUI откроет понятное служебное окно RClone для авторизации.
4. Для сервера используйте `Create/update SFTP remote`: заполните `user@host:port`, путь и метод auth.
5. Если нужен редкий backend, запустите полный `Config`.
6. Запустите `List remotes`.
7. Проверьте `About remote` или `Test remote`.
8. Запустите `Copy endpoint route`, `Upload copy` или `Download copy`.
9. Для проверки используйте `Check endpoint route` или `Check one-way`.

По умолчанию выбран `Portable` config scope: GUI добавляет `--config <ROOT>\config\rclone\rclone.conf` и `--cache-dir <ROOT>\tmp\rclone-cache`.

Переключатель `Portable/System/Custom path` находится в `Configuration`, потому что это выбор `rclone.conf`, а не установка. `System` scope использует обычный `%APPDATA%\rclone\rclone.conf`; `Custom path` принимает явный путь к конкретному `rclone.conf`. `Configuration / Import` копирует system config в portable с backup, `Export` копирует portable config в system с backup, `Doctor` делает read-only проверку active config/remotes/absolute paths. Полная механика описана в [RCLONE_PORTABILITY_RU.md](#).

GUI строит rclone-команды как аргументы subprocess, а preview показывает команду с корректными Windows-кавычками. Пользовательские OAuth-токены и `rclone.conf` не сохраняются в JSON/YAML проекта; portable `rclone.conf` лежит отдельным ignored-файлом в `config\rclone`.

`Config` и `GUI` rclone открываются напрямую в отдельной видимой консоли. `DEBUG` log включайте только временно для локальной диагностики: raw rclone log может содержать

чувствительные детали remote/request.

Перед запуском OAuth/SFTP режима наведите на **ЗАПУСТИТЬ**: tooltip показывает, какое окно откроется, что оставить пустым/default, и какой **remote:path** получится после успешной авторизации.

Для универсального копирования в RClone есть парные панели **SOURCE ENDPOINT** и **TARGET ENDPOINT**. Основной сценарий здесь - remote -> remote, например **drive:Backup -> server:/home/user/backups**; Workbench path можно выбрать вручную как тип endpoint-a, если нужно подключить локальную сторону. SFTP remote собирается в GUI как **user@203.0.113.10:22:/home/user/backups**; password-auth не сохраняет пароль в cache/history, а key/agent-режимы не требуют ввода пароля в проект.

Режимы RClone сгруппированы во вкладки **Передача / Маршрут / Настройки**: на экране остаются только команды выбранной ветки, а активная команда подсвечивается. SFTP **user** и **host/IP** имеют отдельный history dropdown последних успешных операций; кэш хранится в **config\rclone_endpoint_history.json** и не содержит паролей.

Прогресс и медленные каналы

Правый терминал показывает живой вывод. Для RClone прогресс парсится из строк **Transferred**, **Checks**, **Retries**, **Errors**, **ETA** и скорости. GUI обновляет процент, статус, прогресс-бар и текстовые counters. Все transfer-профили rclone используют **--stats 1s --progress**, поэтому на медленных каналах видно, жив ли процесс, сколько ретраев накопилось и куда движется ETA.

Если канал нестабилен, выбирайте **unstable_mobile** или задавайте **Ограничение скорости**: **Без лимита**, **Выбор скорости** или **Своя скорость** со спиннерами, например **10M**. Для спокойных cloud-загрузок можно выбрать профиль **safe_cloud_upload**; OAuth-дефолт теперь Google Drive.

Где лежат данные

- config/gui_settings.yaml** - язык, тема и GUI-настройки без машинно-специфичных source/target путей.
- config/path_history.json** - история и pinned пути.
- config/terminal_commands.json** - история команд правого терминала.
- config/mask_cache.json** - кэш масок.

- `config\rclone_command_cache.json` - кэш/pin/delete конструктора RClone.
- `config\rclone_endpoint_history.json` - история SFTP user/host без паролей.
- `config\sync_pairs.json` - рабочие профили.
- `config\sync_pairs.example.json` - bundled fallback и шаблон.
- `config\sync_presets.json` - группы расширений.
- `logs\` - текстовые логи операций.
- `report\` - отчёты CLI, списки, JSON summaries.
- `workspace\` - временная рабочая зона.
- `Tools\PeaZip\` и `Tools\rclone\` - переносимые внешние инструменты.

Terminal history не сохраняет и не закрепляет команды с очевидными секретными маркерами (`token`, `secret`, `password`, `Authorization`, `Bearer`, `-p...`). Архивные пароли вводятся в маскированное поле, а `-p...` в GUI log редактируется как `-p[REDACTED]`.

Минимальные smoke-команды

```
runtime\python.exe -m py_compile system_core\main.py system_core\auditor_core.py
system_core\services\disk_auditor_service.py system_core\ui_nicegui\app.py
system_core\ui_nicegui\window.py system_core\services\rclone_service.py
runtime\python.exe system_core\ui_nicegui\app.py --smoke
runtime\python.exe system_core\main.py pairs
install\Check-CmdEncoding.cmd
install\Build-Docs-PDF.cmd --dry-run
Tools\rclone\rclone.exe version
Tools\rclone\rclone.exe listremotes
```

Отдельная security-проверка должна завершаться отказом: `runtime\python.exe system_core\ui_nicegui\app.py --host 0.0.0.0 --smoke`.

Полный список проверок: [SMOKE_TEST_CHECKLIST_RU.md](#).

Канонические названия Workbench

Workbench использует единый публичный словарь Audion Image Tools во всех проектах. Кнопки всегда расположены и называются одинаково: **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**.

Сбросить возвращает проектные **input/output** и не удаляет файлы; **Удалить** очищает текущие **Источник** и **Назначение** только после подтверждения. В английском интерфейсе точные названия: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**. Варианты **Цель**, **Очистить**, **Destination** и **Clear** для этих элементов Workbench не используются.

Подробный Справочник По GUI

Этот GUI является рабочей панелью файлового менеджера поверх существующего CLI. Он не дублирует файловое ядро и не хранит отдельную бизнес-логику: операции синхронизации превращаются в запуск **system_core\main.py**, а GUI-only сценарии вроде архивации используют тот же portable runtime и показывают вывод в правой терминальной панели.

Полный стек документации начинается с **..\USER_GUIDE_RU.md**. Справочник всех GUI/CLI команд и параметров: **COMMAND_REFERENCE_RU.md**. Отдельный документ по Workbench, терминалу, кэшам и tooltip: **WORKBENCH_AND_GUI_RU.md**.

Главная модель теперь такая: пользователь выбирает **Источник** и **Назначение**, а затем запускает нужный режим над этим маршрутом. Это могут быть проектные **input/output**, внешний диск, сетевой UNC-путь, смонтированная папка, каталог облачного клиента или любая другая директория, которую видит Windows/Python. Копировать большие папки внутрь проекта не требуется.

Запуск

```
launcher_gui.cmd
```

Desktop-обёртка выбирает следующий свободный локальный порт, если базовый **8080** уже занят другим процессом.

GUI с правым терминалом запускается только на loopback-хостах **127.0.0.1**, **localhost** или **::1**. Запуск на **0.0.0.0**, LAN/VPN-адресе или другом non-loopback host блокируется, потому что веб-панель содержит командный запуск. Для осознанного удалённого режима нужен явный override **AUDION_ALLOW_REMOTE_GUI=1**; это считается RCE surface и не является обычным режимом.

Desktop-wrapper не запрашивает права администратора сам. Если конкретная процедура действительно требует UAC, запускайте wrapper явно с **--elevate** или задавайте

`AUDION_ELEVATE=1`.

Если ruwebview недоступен, серверную часть можно открыть в браузере через:

```
runtime\python.exe system_core\ui_nicegui>window.py --browser
```

Структура окна

Левая колонка:

- текущие **Источник** и **Назначение** с единой pin-иконкой, удалением содержимого, выбором и открытием пути;
- выбор произвольного источника и цели через Windows folder picker;
- создание списка файлов выбранного источника;
- дерево операций;
- параметры выбранного режима, включая фильтры и архивные настройки.

Правая колонка:

- текущий статус;
- прогресс;
- живой терминальный вывод CLI;
- кнопки `Логи`, `Отчёты`, `Последний`, `Настройки`;
- открытие последнего отчёта.

Шапка:

- выбор темы из `config/ui_colors.yaml`;
- переключатель RU/EN;
- кнопка отмены, которая появляется только во время операции.

Все GUI tooltip используют единый тайминг: появление через `1500 ms`, скрывание через `100 ms`, transition duration `100 ms`. Нативный browser `title` отключён, чтобы не возникали два tooltip одновременно; для доступности остаётся `aria-label`. Единый tooltip скруглён, использует фон `RGB(23, 33, 43)`, тонкую рамку и компактный читаемый текст.

Между левой частью и терминалом есть тонкий разделитель: им можно менять ширину правой панели. Выбранная ширина хранится локально в браузерном профиле GUI.

Терминальный вывод

Правая панель является read-only терминалом для живого вывода команд, а не интерактивным shell. Текст можно выделять и копировать мышью.

GUI показывает ANSI-цвета безопасным HTML-рендером: текст экранируется, поддерживаются только разрешённые SGR-коды, а управляющий мусор отбрасывается. Кириллица из Python-команд идёт в UTF-8, а вывод Windows-утилит декодируется через fallback-цепочку для OEM/cp866/cp1251.

Видимый терминал переносит длинные строки без нижней горизонтальной прокрутки и держит до 2000 строк текущей операции. Полные логи и большие списки сохраняются на диске обычным UTF-8 текстом без ANSI escape-последовательностей.

Командная строка под терминалом хранит историю до 200 записей в

`config\terminal_commands.json`,

поддерживает пины, выбор Shell и рабочей папки. Поле команды при запуске остаётся пустым: прошлые команды выбираются из истории. Кнопка

История

очищает незакреплённую историю, **Кэш** полностью сбрасывает history/pinned/last command, сохраняя Shell и рабочую папку. Команды, похожие на содержащие секреты (

`token`,

`secret`,

`password`,

`Authorization`,

`Bearer`,

`-p...`

и похожие маркеры), не сохраняются и не закрепляются.

Рабочий маршрут

Верхняя панель показывает две текущие рабочие папки:

- **Источник** — откуда читать файлы для списка, синхронизации, аудита и архивации.
- **Назначение** — куда писать результаты операций, архивы и часть отчётов.

В каждой плашке есть три компактные иконки:

- delete очищает содержимое текущей папки или удаляет выбранный файл-источник;
- единая pin-иконка закрепляет путь, а в активном состоянии снимает закрепление;
- picker выбирает другую папку через системный диалог.

Сам путь является выпадающим элементом: нажмите на строку пути, чтобы открыть широкий список кэша. Закреплённые записи идут сверху и отмечаются pin-иконкой.

Кнопки **Источник** и **Назначение** в нижнем ряду открывают текущие пути. **Добавить файл...** назначает один файл текущим источником без копирования. **Сбросить** возвращает маршрут на

проектные `input/output`, не удаляя файлы и сохраняя закреплённые записи. Иконка удаления в строке очищает выбранную папку или удаляет выбранный файл; внешний источник требует подтверждения. Кнопка `Удалить` после отдельного подтверждения очищает текущие `Источник` и `Назначение`, в том числе внешние маршруты.

Кнопка `Список` строит предпросмотр найденных файлов для текущего источника. Если выбран профиль из `Сохранённых профилей` или ручная операция с фильтром расширений, список проходит через тот же фильтр/preset/mask, который будет использовать операция. Если активного фильтра нет, показываются все файлы источника. Полный список сохраняется в `report`, а терминал показывает быстрый предпросмотр.

В операциях с расширениями один общий набор масок переключается вкладками `ВКЛЮЧИТЬ` и `ИСКЛЮЧИТЬ`. Вкладка `ВКЛЮЧИТЬ` задаёт include/mask-globs, вкладка `ИСКЛЮЧИТЬ` хранит отдельную конфигурацию исключений и убирает выбранные маски после применения профиля/include-фильтра. Исключения уходят в CLI как `--exclude-globs`.

В `ТРАНСФЕР ДАННЫХ`, `ПАПКА В ПАПКУ` и в сохранённых профилях над списком расширений закреплена одинаковая рабочая поляна: Pin/Unpin, удаление набора из кэша, очистка выбора, выпадающий кэш и четырёхстрочная сводка выбранных групп и точечных масок для `ВКЛЮЧИТЬ` и `ИСКЛЮЧИТЬ`. При прокрутке длинных групп эта сводка остаётся сверху, а сами чекбоксы уходят под неё.

Сетки расширений выравниваются по строгому правилу: три плитки в строке имеют одинаковую ширину, две плитки делят всю строку пополам, одиночная или очень длинная группа занимает всю ширину. Внутри плитки 1-3 чекбокса идут вертикально, а с 4 чекбоксов включаются две внутренние колонки. Поэтому блоки вроде `Видео` и `Аудио` держат ровные вертикальные линии и не выглядят случайным набором разных карточек.

`Маски` - ручной режим для одноразовых или часто повторяемых наборов расширений и одновременно генератор glob-строки для внешних программ. Маски вводятся через запятую без учёта регистра в четырёхстрочное поле: `jpg, .PNG, *.webp` нормализуется в `*.jpg, *.png, *.webp`. Кнопка нормализации превращает ввод в готовые glob-маски, которые можно использовать не только здесь, но и в Total Commander или скриптах. Наборы масок сохраняются в `config\mask_cache.json`; их можно закреплять, откреплять, удалять и выбирать из списка. Чекбоксы расширений снизу сразу добавляют выбранные расширения или группы в поле масок, а выбор `ИСКЛЮЧИТЬ` вычитает эти маски из итоговой строки.

Количество профилей и presets не ограничено кодом: practically это обычные JSON-списки/словари. Профиль называется полем `name` в `sync_pairs.json`; фильтрационный preset называется ключом в `sync_presets.json`. Раздел `Сохранённые профили` собирается из этих

имён динамически. При запуске профиля из GUI текущие верхние **Источник** и **Назначение** подставляются поверх путей из профиля.

Текущий маршрут при запуске GUI сбрасывается на проектные **input/output**, а кнопка **Сбросить** возвращает Workbench туда же без удаления файлов. История путей хранится отдельно в локальном **config\path_history.json**: до 100 записей, счётчик использования, время последнего выбора и флаг pinned. Portable **config\gui_settings.yaml** не сохраняет машинно-специфичные source/target пути.

Основные режимы

В ROOT находятся: **ТРАНСФЕР ДАННЫХ**, **ПАПКА В ПАПКУ**, **Маски**, **Сохранённые профили**, **Архивация**, **Manifest / Diff**, **ПОДГОТОВКА** и **Обслуживание**.

ПОДГОТОВКА - компактный список пробных запусков, compare-only отчётов и quarantine-вариантов для ручных путей и сохранённых профилей. Рабочие профили открываются сразу без промежуточного child-меню.

BACKUP-MIRROR dry run

Показывает, что будет сделано выбранной backup policy:

- какие файлы будут скопированы из **source** в **target**;
- какие файлы будут обновлены;
- какие target-only файлы будут перенесены в карантин или удалены при **MIRROR_HARD**.

Файлы на диске не меняются.

BACKUP-MIRROR

Mirror-режим бэкапа. По умолчанию это hard mirror: копирование/обновление, проверка, затем удаление target-only файлов, если ранние фазы успешны.

MIRROR_SAFE остаётся отдельной policy, если нужен карантин вместо direct delete.

BACKUP-MIRROR карантин

Дополнительный mirror-режим. Копирует и обновляет так же, как **BACKUP-MIRROR**, но target-only файлы переносит в **_audion_quarantine\YYYYMMDD_HHMMSS_microseconds**, сохраняя относительные пути.

One-Way dry run

Показывает, какие новые и изменённые файлы будут скопированы из **source** в **target**.

Target-only файлы не удаляются.

Односторонняя

Копирует новые и изменённые файлы из **source** в **target**.

Этот режим подходит для мягкого export/copy, когда **target** может содержать дополнительные файлы.

Compare only

Строит отчёт различий без изменений на диске.

Используйте его перед важными или большими запусками, если нужно сначала понять расхождение папок.

Two-Way dry run / Two-Way sync

Сценарий для сведения двух рабочих папок. Копирует отсутствующие или более новые файлы в обе стороны и не делает автоматических удалений.

Manifest / Diff

Manifest / Diff работает поверх текущего маршрута Workbench **Источник -> Назначение**. Сначала выбирается фильтр расширений: пустой выбор означает все файлы, выбранные группы и точечные расширения ограничивают расчёт. Затем можно нажать **ТОЛЬКО DIFF**, чтобы получить списки относительных путей в **report**, или сразу выполнить **DIFF + ONE-WAY SYNC** / **DIFF + BACKUP-MIRROR**.

Отдельные кнопки **Создать источник**, **Создать цель**, **Проверить источник** и **Проверить цель** создают или проверяют служебные **__CHECKSUMS__.b3**. Они полезны для одиночного integrity-аудита, но не обязательны для Diff + One-Way.

Архивация

Архивация работает с текущим маршрутом: читает элементы из выбранного источника и создаёт результаты в выбранной цели.

Поддерживаемые форматы:

- ZIP;
- 7Z;
- SFX `.exe`;
- TAR;
- TAR.GZ;
- TAR.ZSTD через `zstd.exe` из portable PeaZip;
- дополнительный TAR.LZ4 через отдельный `lz4.exe`; backend ставится кнопкой `INSTALL LZ4` в панели `Архивация` или шагом `[06] LZ4` в `builder_main.cmd`.

`Куда класть` выбирается inline radio-чипами: `ЦЕЛЬ` пишет архивы прямо в текущую цель, `ЦЕЛЬ/<имя>` создаёт подпапку для каждого исходного элемента. `Проверить архивы после сжатия` тестирует каждый результат перед возможным удалением исходников; `Удалить исходные файлы и папки` срабатывает только после успешного создания и, если включено, успешной проверки.

Шифрование включается явно. `Не шифровать` — режим по умолчанию. Если выбранный backend не поддерживает шифрование или сам формат сейчас недоступен в portable-комплекте, соответствующий чекбокс формата затемняется, чтобы интерфейс не обещал невозможное. Поле пароля маскируется, а `-p...` редактируется в GUI log как `-p[REDACTED]`. Сам пароль всё равно передаётся CLI-архиватору как аргумент процесса, поэтому не храните его в командной строке терминала и не используйте публичные машины.

SFX-режим умеет задавать путь автоэкстракции через Windows-переменные окружения. В UI показывается абсолютное превью для текущей машины, но внутрь SFX пишется переносимый путь с переменной, например `%LOCALAPPDATA%\Audion\<имя>`. После SFX-компрессии можно дополнительно упаковать `.exe` в ZIP/7Z/ZSTD/TAR/GZ контейнер с уровнем 0, чтобы файл было проще отправлять там, где `.exe` блокируется.

ТРАНСФЕР ДАННЫХ

Раньше разделов было два: `ОПЕРАЦИИ ПО СЕТИ` для LAN, SMB и UNC-путей, и `ОБЛАЧНЫЕ ОПЕРАЦИИ` для remotes вроде `drive:path`. Чтобы выбрать кнопку, надо было сперва знать, какой инструмент куда дотягивается. Теперь это один раздел `ТРАНСФЕР ДАННЫХ`, и папка, шара, сервер и облако равноправны.

Выбирают действие, а не инструмент. Операций пять: **Односторонняя**, **Зеркало**, **Перенести**, **Двусторонняя**, **Сверить**. Движок под каждую пару выбирается сам и назван в окне отдельной строкой — Roboscopy на папки и шары, RClone на облака, двустороннюю синхронизацию и сверку по хешу.

Зеркало и **Односторонняя** — разные операции, а не одна с галочкой удаления. У зеркала назначение — копия и ничего своего не имеет: изменилось на эталоне, разъехалось по репликам, включая удаления. У односторонней у назначения своя жизнь, и делать его точной копией источника значило бы стереть то, что там лежит.

Назначение — список, а не одно поле: разложить с эталонной машины на четыре реплики можно одним запуском. Вся раскладка собирается до старта, поэтому невозможная пара видна раньше, чем тронется первый байт.

Сначала упаковать собирает источник в архивы и отправляет их вместо россыпи файлов. На мелких файлах по сети это единственное, что реально помогает: замерено 24 МБ/с одним файлом против 0.46 МБ/с тремястами мелких, при загрузке процессора около 20% — время уходит не на передачу, а на то, что сеть здороваается с каждым файлом отдельно.

Пробный прогон есть у всех операций и ничего не пишет.

Подробности — операции, выбор движка, маски, коды возврата Roboscopy, потолок удалений — в [TRANSFER_RU.md](#).

ОБЛАЧНЫЕ ПОДКЛЮЧЕНИЯ И ХРАНИЛИЩЕ

ОБЛАЧНЫЕ ОПЕРАЦИИ находится в ROOT сразу после сетевого раздела. Это отдельный cloud/remotes слой для rclone backend-ов, включая Yandex Disk native **yandex:**, Google Drive, SFTP, WebDAV и S3-compatible remotes.

Вверху раздела всегда находится компактная вкладочная навигация **Передача / Маршрут / Настройки**. **Передача** показывает только upload/download/check для обычного **remote:path**, **Маршрут** - endpoint route, а **Настройки** - installer, config, OAuth/SFTP, GUI и диагностики. Блок **RClone config** появляется только на вкладке **Настройки** в режиме **Настройка**, поэтому **Версия**, **Список remote**, **GUI**, OAuth и SFTP не тянут за собой лишний installer/config-manager. Линейка **Installer RClone** содержит только кнопки **System** и **Portable**. **System** запускает **install\Install-System-Rclone.cmd** и ставит user-scope RClone в **%LOCALAPPDATA%\Programs\rclone**; **Portable** запускает **install\Install-Portable-Rclone.cmd** и обновляет только **Tools\rclone**. Оба installer-а не перезаписывают

`rclone.conf`. Режимы `Config` и `GUI` открываются напрямую в отдельной видимой консоли без промежуточного shell-string; обычные команды стримятся в правый терминал.

Линейка `Configuration` содержит выбор config scope `Portable`, `System`, `Custom path`, поле custom config path и кнопки `Import`, `Export`, `Doctor`. `Custom path` - это путь к конкретному `rclone.conf`, не место установки. Импорт копирует system `rclone.conf` в portable config с backup, экспорт копирует portable config в system config с backup, doctor read-only показывает active config, remotes и абсолютные пути. По умолчанию выбран `Portable` scope, поэтому команды получают `--config <ROOT>\config\rclone\rclone.conf` и `--cache-dir <ROOT>\tmp\rclone-cache`.

Режимы RClone не показаны одним длинным dropdown. Сначала выбирается ветка вкладкой: `Передача`, `Маршрут` или `Настройки`; ниже видны только команды этой ветки. Активная команда подсвечивается приглушённым success-цветом. Под кнопками остаётся короткая pipeline-сводка активного режима, а tooltip каждой кнопки объясняет маршрут вида `Источник -> remote:path -> rclone copy`, `remote -> remote -> rclone copy` или `remote name + backend -> OAuth window -> remote:path`.

`Ограничение скорости` растянуто отдельной строкой под флагами RClone. Внутри есть `Без лимита`, `Выбор скорости` и `Своя скорость` со спиннерами; одновременно активен только один источник значения, поэтому preset/custom не накладываются.

Upload использует текущий Workbench `Источник` и строит `rclone copy <Источник> <remote>:<path>`. Download использует текущее Workbench `Назначение` и строит `rclone copy <remote>:<path> <Назначение>`. Проверка использует `rclone check <Источник> <remote>:<path> --one-way` и пишет combined report в `logs`.

Для универсальных маршрутов есть парные toggle-панели `SOURCE ENDPOINT` и `TARGET ENDPOINT`: основной сценарий - сохранённый `remote:path` на обеих сторонах, например `drive:Project -> server:/home/user/backups` или `server:/home/user/backups -> drive:Restore`. Workbench Source/Target остаются явными вариантами endpoint type для локальных включений.

Авторизация вынесена в toggle-панель `REMOTE`. SFTP собирается как `user@host:port` с выбором `ssh-agent`, `password` или `key_file`; OAuth/provider backend-ы открываются отдельным понятным окном RClone wizard. Для редких backend-ов остаётся полный `Config`.

В режиме `Провайдеры OAuth` дефолтный provider - Google Drive, а `Имя remote` по умолчанию `drive`. Поле автоматически следует за выбранным provider-ом, пока оно пустое или содержит

дефолтное имя вроде `drive`, `yandex`, `dropbox`. Если пользователь ввёл своё имя, GUI его не перезаписывает.

Tooltips у `Remote name`, `Провайдер`, SFTP-полей и `ЗАПУСТИТЬ` обязаны объяснять точное действие: какое окно откроется, что вводить в prompts, где оставить Enter/default, и какой `remote:path` получится после success. Для Google Drive обычный путь: remote name `drive`, `Client ID`/`Client Secret` пустые, advanced config default/No, auto config/browser Yes, затем разрешить доступ в браузере.

SFTP `user` и `host/IP` имеют history dropdown последних успешных операций. Этот cache лежит в `config\rclone_endpoint_history.json`, не содержит паролей и очищается кнопкой очистки в RClone auth-панели. Примеры host в документации используют reserved example IP `203.0.113.10`.

Внизу формы есть компактный конструктор: exe, route, config/cache, source/target, log и полный command preview мелким шрифтом. Шаблоны команд можно закреплять, откреплять и удалять из кэша; OAuth tokens и SFTP password не сохраняются в project JSON/YAML. Portable `rclone.conf` хранится как ignored-файл в `config\rclone`.

`rclone_log_level=DEBUG` предназначен только для локальной диагностики: raw rclone log может содержать чувствительные детали remote/request. В обычных операциях оставляйте `INFO`.

На медленных каналах верхний progress bar/status обновляется из штатных rclone stats: `Transferred`, `Checks`, `Retries`, `Errors`, скорость и ETA. Правый терминал при этом показывает полный поток rclone без сокращения.

Сохранённые профили

Профили лежат в `config\sync_pairs.json`, фильтры - в `config\sync_presets.json`.

В GUI сохранённые профили теперь являются прямыми действиями: выбор `dev_backup`, `docs_backup` и других профилей сразу открывает форму запуска. Их `пробно`, `сравнить` и `карантин` варианты находятся в `ПОДГОТОВКА`.

Текущая договорённость:

- `*_backup` examples - hard Backup mirror по умолчанию;
- `MIRROR_SAFE` доступен отдельной quarantine-кнопкой или через явный `operation_policy`.

Старый ключ `dry_run_default` игнорируется с deprecation warning. Используйте `operation_policy: copy_update`, `mirror_safe` или `mirror_hard`.

Фильтр расширений

В GUI у профилей и ручных режимов есть блок расширений. Это не маленькая галочка, а широкий браузер типов файлов поверх текущего источника.

У `ТРАНСФЕР ДАННЫХ`, `ПАПКА В ПАПКУ` и сохранённых профилей блок устроен одинаково: сверху закреплённая сводка выбора и кэш наборов, ниже вкладки `ВКЛЮЧИТЬ` / `ИСКЛЮЧИТЬ` с частотной группировкой документов, картинок, архивов, видео, аудио, приложений, разработки и прочего.

Пустой выбор означает:

- для профиля - использовать фильтры из `config\sync_pairs.json` и `config\sync_presets.json` как есть;
- для ручного режима - не добавлять runtime `--mask-globs`.

Выбранные группы разворачиваются в список масок только для текущего запуска. GUI не переписывает YAML-конфиги из блока расширений.

Типовые семьи фильтров:

- документы и текст;
- разработка и конфиги;
- приложения и Windows-файлы;
- архивы и образы;
- аудио, видео и графика;
- временные и служебные файлы;
- ручные glob-маски вроде `*.docx`, `*.pdf`, `*.zip;*.7z`.

Один и тот же источник можно сначала просмотреть списком, затем отфильтровать, сравнить, синхронизировать, забэкапить или упаковать в архив.

Отчёты

Основные артефакты:

- `output\` - отчёты compare/sync/backup;

- выбранная **Назначение** - архивы и рабочие результаты GUI-архивации;
- `report\` - GUI/run-артефакты;
- `logs\` - логи GUI и launcher-слоя.

Кнопка `Последний отчёт` в правой панели пытается открыть самый свежий человекочитаемый отчёт из `output\` или `report\`.

Внешний вид

Настройки GUI разделены на два файла:

- `config\ui_colors.yaml` — палитры, CSS-токены, шрифты и темы;
- `config\gui_settings.yaml` — стартовый язык, выбранная тема, emoji-флаг и GUI-only настройки без машинно-специфичных source/target путей.

Внутренний стандарт:

- тема `code_dark` по умолчанию;
- переключение тем из шапки с полным reload интерфейса;
- компактные кнопки без заливки;
- приглушённые границы;
- описание режима справа от кнопки;
- терминал справа как продолжение старого CMD/FZF workflow.

Русские подписи в рабочей панели намеренно короткие: `Логи`, `Отчёты`, `Последний`, `Настройки`, `Источник`, `Назначение`, `Список`. Длинный смысл остаётся в tooltip, README, соседнем описании команды или в отчёте, чтобы кнопки не ломали раскладку на FullHD и ноутбучных экранах.

Cleanup и структура папок

`install\init_folders.cmd` создаёт все управляемые папки, которые ожидают CLI и GUI:

`input`, `output`, `logs`, `report`, `workspace`, `data`, `runtime`, `wheelhouse`, `release`, `._runtime`, `system_core`, `install`, `licenses`.

`input` и `output` остаются удобными дефолтами и staging-зонами, но больше не являются обязательной рабочей моделью. Для больших папок выбирайте внешний источник и внешнюю цель в верхней панели.

`cleanup_project.cmd` очищает только содержимое управляемых рабочих/runtime папок и кэши. Исходники, документация, `config\sync_pairs.json`, `config\sync_presets.json`, `config\ui_colors.yaml`, `config\gui_settings.yaml` и лицензии остаются на месте.

Безопасная лестница

Для важных папок:

1. `Compare only`.
2. Dry-run выбранного режима.
3. Проверка отчёта, quarantine candidates и возможных hard-delete candidates.
4. Реальный запуск.

Для обычного бэкапа чаще всего достаточно:

1. `Односторонняя` / `COPY_UPDATE`.
2. `BACKUP-MIRROR`, когда нужен настоящий mirror с direct delete после успешных copy/verify фаз.

Канонические названия Workbench

Workbench использует единый публичный словарь Audion Image Tools во всех проектах. Кнопки всегда расположены и называются одинаково: **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**.

`Сбросить` возвращает проектные `input/output` и не удаляет файлы; `Удалить` очищает текущие `Источник` и `Назначение` только после подтверждения. В английском интерфейсе точные названия: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**. Варианты `Цель`, `Очистить`, `Destination` и `Clear` для этих элементов Workbench не используются.